AI505 – Optimization

Answers to Obligatory Assignment 1, Spring 2025

Simon Holm, Johannes Rothe

Always insert a `\newpage` after every Task. Remove the content of this page.

Template to write models (see latex source)

$$\max \quad \sum_{j=1}^{n} c_j x_j \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} a_{ij} x_j \geq b_i, \qquad i = 1, \ldots, m \tag{2}$$

$$x_j \geq 0, \qquad j = 1, \ldots, n \tag{3}$$

Template for algorithm pseudocode (see latex source):

Algorithm 1: How to write algorithms

```
Data: this text
Result: how to write algorithm with LaTeX2e
initialization;
while not at end of this document do
    read current;
    if understand then
        go to next section;
        current section becomes this one;
    else
        go back to the beginning of current section;
```

Template for source code inclusion:

```
import numpy as np
```

Template for figure inclusion (see latex source)

1 Task 1

A robot must move from a given start position to a given goal position in a 2D environment while avoiding obstacles and producing a smooth trajectory.

$$\{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^2$$

Lets fix 2 points, that is the starting point x_1 and the goal point x_n

$$x_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad x_n = \begin{pmatrix} 100 \\ 100 \end{pmatrix}$$

For this, lets define a class in python to handle the search space.

```
class search_space():
    def __init__(self, start, goal):
        self.start = start
        self.goal = goal
        self.obstacles = []
        self.trajectory = []
```

1.0.1 Straight path

For a straight path we can generate a path for finding x.

The straight path can be easily found by

Algorithm 2: Straight path

Input: start $x_1 = (1, 1)$, goal $x_n = (100, 100)$, steps $s = 20$
Output: trajectory T

$d \leftarrow x_n - x_1$;
$T \leftarrow []$;
for $t \in$ *linspace*$(0, 1, s)$ **do**
  $p \leftarrow x_1 + t \cdot d$;
  append p to T;
return T;

1.0.2 Obstacles

For creating obstacles, lets define a new class `obstacle` and an `add_obstacle` function to the `search_space` class

```
class circular_object():
    def __init__(self, center: np.array, diameter: float):
        self.center_point = center
        self.diameter = diameter

class search_space():
    # ...
    def add_obstacle(self, object: circular_object):
        self.obstacles.append(object)
```

This way its easy to create objects within the search space

```
search = search_space(x0, xn)

object1 = circular_object(np.array([40,30]), 10.0)
search.add_obstacle(object1)
object2 = circular_object(np.array([50,55]), 7.0)
search.add_obstacle(object2)
object3 = circular_object(np.array([60,80]), 15.0)
search.add_obstacle(object3)
```

1.0.3 Visualisation

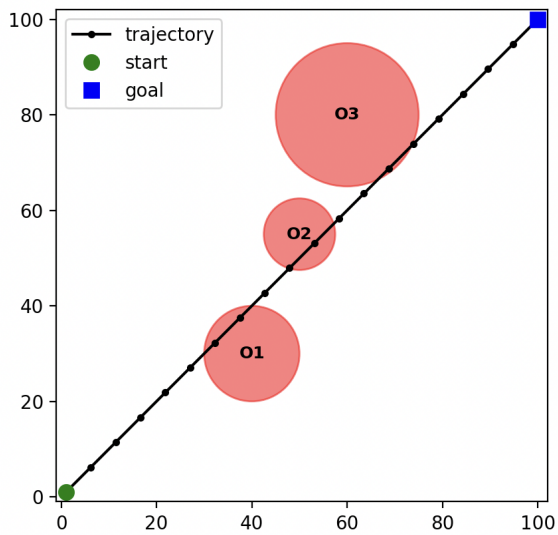

Figure 1: Linear path with three obstacles. `[circular_object(c=[40 30], d=20.0), circular_object(c=[50 55], d=15.0), circular_object(c=[60 80], d=30.0)]`

1.1 Objective Function

The objective function is defined as

$$f(\mathbf{x}) = f_L(\mathbf{x}) + \lambda \cdot f_S(\mathbf{x}) + \mu \cdot f_O(\mathbf{x})$$

or in python

```
class search_space():
    # ...
    def obj_func(self, x, lam, mu, alpha):
        def f(x):
            traj = x.reshape(-1, 2)
            return (self.pathlength(traj)
                    + lam * self.smoothness(traj)
                    + mu * self.avoidance(traj, alpha))

        return f(x), grad(f)(x)
```

1.1.1 Path length

The path length function is defined as

$$f_L(\mathbf{x}) = \sum_{i=1}^{n-1} \|x_{i+1} - x_i\|^2$$

or in python

```
class search_space():
    # ...
    def pathlength(self):
        path = self.trajectory
        return sum(np.linalg.norm(path[i+1] - path[i])
                   for i in range(len(path) - 1))
```

1.1.2 Smoothness

The smoothness function is defined as

$$f_S(\mathbf{x}) = \sum_{i=2}^{n-1} \|x_{i+1} - 2x_i - x_{i-1}\|^2$$

or in python

```
class search_space():
    # ...
    def smoothness(self):
        x = self.trajectory
        return sum(np.linalg.norm(x[i+1] - 2*x[i] - x[i-1])
                   for i in range(1, len(x) - 1))
```

1.1.3 Object avoidance

The obstacle avoidance function is defined as

$$f_O(\mathbf{x}) = \sum_{i=1}^{n} \phi(x_i)$$

$$\phi(\mathbf{x}_i) = \exp\left(-\alpha \left(d(\mathbf{x}_i)^2 - r^2\right)\right)$$

or in python

```
class search_space():
    # ...
    def avoidance(self, x, alpha):
        penalty = 0
        for xi in x:
            penalty += sum(anp.exp(-alpha * (
                anp.linalg.norm(xi - obs.center_point)**2 - obs.radius**2))
                    for obs in self.obstacles
            )
        return penalty
```

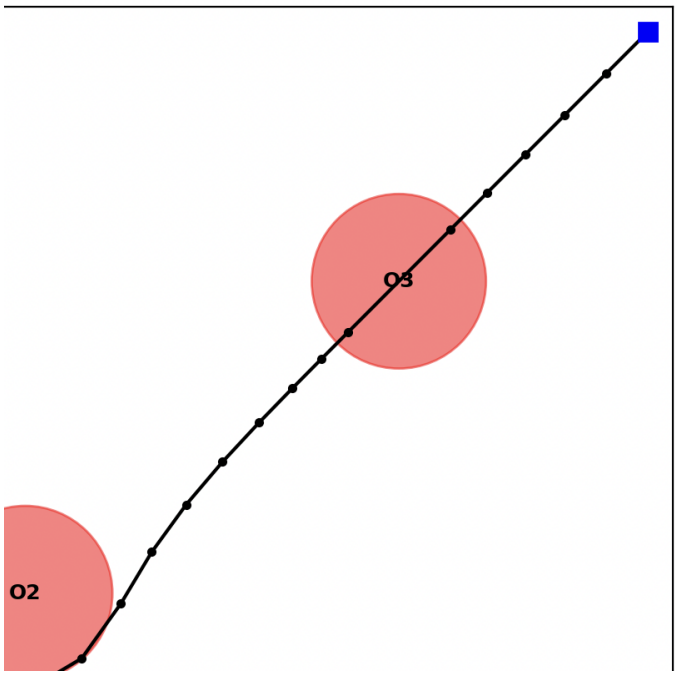

Figure 2: der sker en ting med ham her

1.2 Optimization Algorithms

1.3 Comparison

2 Task 2

A Appendix

Here we shoudl talk about AI use in the project

in your you have to add an Appendix where you declare to which extent you have used AI tools. The Appendix must be present also if you did not use AI and hence declare so.

if you do not declare but the teachers have suspects that you used AI tools, you will be reported for exam cheating

if you use AI, the teachers will be allowed to take this in consideration and to lower consequently the final grade if they suspect that the learning goals have not been achieved.

In declaring the use you can share the conversation you had with the tools and provide the link or copying it in the Appendix. The Appendix is not subjected to page limits.